

Tema 8.

Clasificación temática de textos

Práctica I: Clasificación binaria de mensajes SMS

Prácticas | Programación para PLN | Universidad Pontificia Comillas

Proyecto de práctica

Encargo

LexiData Solutions quiere construir un primer detector automático que diferencie mensajes SMS legítimos de mensajes no deseados. El objetivo no es desplegar un sistema real de seguridad, sino comprender de forma controlada cómo se transforma texto breve en variables numéricas y cómo se evalúa un clasificador binario.

El encargo consiste en preparar un flujo completo de clasificación: cargar el corpus, revisar la distribución de clases, limpiar los textos, dividir los datos, entrenar un modelo con TF-IDF y un clasificador lineal, evaluar los errores y probar mensajes nuevos.

Tu papel

Actúa como analista junior de PLN. Tu tarea es construir un cuaderno reproducible que permita a otra persona entender qué datos se han usado, qué transformación textual se ha aplicado, qué aprende el modelo y dónde puede fallar.

No basta con obtener una accuracy alta: debes explicar las métricas y razonar el efecto de los falsos positivos y falsos negativos en un detector de spam.

Actividad de práctica: Clasificación de SMS *spam/ham*

Este documento contiene 10 ejercicios independientes y acumulativos para practicar la clasificación supervisada de textos con un flujo completo de aprendizaje automático aplicado a PLN.

Las actividades reproducen las operaciones trabajadas en la unidad: carga del corpus, análisis exploratorio, preprocesamiento, división entrenamiento-prueba, vectorización TF-IDF, entrenamiento, evaluación, prueba con textos nuevos e interpretación prudente de las predicciones.

Corpus de trabajo

El corpus de trabajo es SMS Spam Collection, formado por 5.574 mensajes SMS en inglés etiquetados como ham o spam.

La práctica trabaja con textos breves, vocabulario informal y una distribución de clases desbalanceada. Estas características obligan a interpretar las métricas por clase, no solo el rendimiento global.

Ejercicio 1. Carga del corpus SMS Spam Collection

Importa el conjunto SMS Spam Collection con pandas desde esta dirección, indicando la codificación latin-1:

```
import pandas as pd
url = ('https://raw.githubusercontent.com/mohitgupta-omg/'
      'Kaggle-SMS-Spam-Collection-Dataset-/master/spam.csv')
data = pd.read_csv(url, encoding='latin-1')
```

Si no puedes descargarlo así, el fichero spam.csv se descarga también desde la web, en la sección de Prácticas del Tema 8. Después, crea una tabla de trabajo con las columnas necesarias para el problema.

Comprueba que cada fila contiene un mensaje SMS y una etiqueta de clase. Renombra las variables de forma clara para trabajar con texto y etiqueta.

Ejercicio 2. Limpieza inicial de columnas

Elimina las columnas auxiliares que no aportan información al problema de clasificación.

Muestra las primeras filas del dataset depurado y explica qué representa cada columna conservada.

Ejercicio 3. Análisis exploratorio de datos

Comprueba si hay valores ausentes, calcula el número total de mensajes y representa la distribución de las clases ham y spam.

Comenta si el problema está equilibrado o desbalanceado y qué efecto puede tener sobre la evaluación.

Ejercicio 4. Preprocesamiento de mensajes

Diseña una función o bloque de preprocesamiento que normalice los SMS: limpieza de caracteres no alfabéticos, minúsculas, tokenización, eliminación de stopwords y lematización.

Aplica el proceso al corpus y compara algunos mensajes antes y después de la limpieza.

Ejercicio 5. División entrenamiento-prueba

Divide los textos y sus etiquetas en conjuntos de entrenamiento y prueba usando una proporción 80/20.

La partición debe mantener la proporción de clases. Indica cuántos mensajes quedan en cada conjunto.

Ejercicio 6. Vectorización TF-IDF

Convierte los mensajes en una representación numérica mediante TF-IDF.

Incluye unigramas y bigramas y aplica un umbral mínimo de frecuencia. Explica por qué TF-IDF puede ser más informativo que un conteo bruto de palabras.

Ejercicio 7. Entrenamiento del clasificador

Entrena un clasificador lineal adecuado para texto usando la representación TF-IDF.

Organiza el flujo en un pipeline para que la vectorización y el modelo se ejecuten de manera reproducible.

Ejercicio 8. Evaluación del rendimiento

Evalúa el modelo sobre el conjunto de prueba mediante informe de clasificación y matriz de confusión.

Interpreta precision, recall y f1-score para las clases ham y spam, prestando especial atención a los falsos negativos de spam.

Ejercicio 9. Clasificación de SMS nuevos

Inventa tú dos mensajes SMS nuevos, en inglés (el modelo se ha entrenado con mensajes en inglés): uno que parezca una comunicación cotidiana y otro con rasgos típicos de spam.

Obtén la predicción del modelo para ambos y comenta si el resultado es razonable.

Ejercicio 10. Análisis de términos influyentes

Elige el mensaje nuevo clasificado como spam y revisa qué palabras o n-gramas han contribuido más a esa decisión.

Explica la predicción con cautela: las contribuciones del modelo ayudan a interpretar, pero no equivalen a una comprensión humana del mensaje.

Actividad final de reflexión

1. ¿Por qué una accuracy alta puede ser engañosa en un corpus desbalanceado?
2. ¿Qué diferencia práctica hay entre un falso positivo y un falso negativo en detección de spam?
3. ¿Por qué TF-IDF puede reducir el peso de palabras muy frecuentes pero poco informativas?
4. ¿Qué aportan los bigramas frente a los unigramas en mensajes breves?
5. ¿Qué límites tiene explicar una predicción mediante pesos de palabras?

Rúbrica de evaluación

Se valora la construcción reproducible del flujo de clasificación textual, la interpretación crítica de las métricas y la capacidad para explicar predicciones sin convertir el modelo en una caja negra absoluta.

Criterio	Excelente	Adecuado	En proceso
Carga y análisis del dataset	Carga correctamente el corpus, identifica variables, tamaño, clases, valores ausentes y distribución de etiquetas, y comenta sus implicaciones.	Carga el corpus y revisa los aspectos principales, aunque la interpretación es parcial o poco conectada con el problema.	No consigue preparar el dataset o no distingue claramente textos, etiquetas y distribución de clases.
Preprocesamiento textual	Aplica una limpieza reproducible y justifica las decisiones de normalización, tokenización, stopwords y lematización.	Aplica un preprocesamiento básico, aunque con alguna decisión poco explicada o con resultados presentados de forma incompleta.	No obtiene una representación textual limpia o aplica pasos sin comprender su efecto.
Extracción de características y entrenamiento	Construye una representación TF-IDF adecuada, entrena el clasificador y explica la función de cada componente del pipeline.	Entrena un modelo funcional, aunque la explicación de TF-IDF, n-gramas o clasificador es limitada.	No logra entrenar un modelo coherente o confunde representación textual y clasificación.
Evaluación e interpretación	Evalúa con métricas pertinentes, interpreta precisión, recall, f1-score y matriz de confusión, y detecta errores relevantes.	Presenta métricas básicas, aunque la lectura de los errores o de las clases es incompleta.	Se limita a mostrar resultados sin interpretarlos o interpreta incorrectamente las métricas.
Prueba y explicación de predicciones	Prueba textos nuevos, razona la predicción y analiza términos influyentes con prudencia, sin confundir correlación con explicación causal.	Prueba textos nuevos y ofrece una explicación general, aunque poco apoyada en evidencias del modelo.	No verifica el modelo con textos nuevos o atribuye las predicciones a causas no observables.
Claridad y reproducibilidad	Entrega un cuaderno ordenado, ejecutable y comentado, con resultados visibles y conclusiones breves.	Entrega un trabajo comprensible, aunque con pequeños problemas de orden, ejecución o presentación.	El trabajo no es reproducible o carece de una secuencia clara de ejecución.